SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom' s Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I AATHISRI P(192521252)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
AATHISRI(192521252)

1. Online Banking System – ECP and BVA

Introduction:

The online banking system experiences failures during fund transfers, especially at minimum and maximum limits. Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) can improve test coverage.

1. Equivalence Class Partitioning (ECP)

Divide transfer amounts into valid and invalid classes. Assume the permitted transfer range is ₹100 to ₹50,000.

Test case	Input amount	Expected result
Below minimum	₹99	Reject
Minimum value	₹100	Accept
Valid amount	₹5,000	Accept
Maximum value	₹50,000	Accept
Above maximum	₹50,001	Reject
Zero amount	₹0	Reject
Negative amount	-₹100	Reject

2. Boundary Value Analysis (BVA)

Test values immediately below, at, and above both limits: ₹99, ₹100, ₹101 and ₹49,999, ₹50,000, ₹50,001.

3. Evaluation of Random Testing

Random testing may identify unexpected errors, but it can miss boundary defects because critical values may not be selected. Therefore, it should supplement systematic testing rather than replace it entirely.

4. Improved Testing Strategy

- Apply ECP and BVA to transfer amounts.
- Test insufficient balance, invalid account numbers and network failures.
- Verify balance deduction and credit after transfers.
- Automate repeated boundary tests.
- Perform regression testing after fixes.

Conclusion: Combining systematic black-box techniques with supplementary random testing improves defect detection, transaction accuracy and banking reliability.

2. Healthcare Application – White-Box Testing and Coverage

Introduction: Critical defects after deployment indicate insufficient code coverage and inadequate test design. White-box testing examines internal program logic and execution paths.

1. Construct a Control Flow Graph (CFG)

Consider a healthcare module that validates a patient's details and processes an appointment.

Start
Validate patient details
Details valid?

Yes

Process appointment
End

No

Display error
End

2. Coverage Criteria

- Statement coverage: Every executable statement is run at least once.
- Branch coverage: Both true and false outcomes of each decision are tested.
- Path coverage: Different execution paths are tested. Complete path coverage may be impractical when loops create many paths.

3. Missing Paths

The previous testing may have covered only valid patient details, missing invalid details, unavailable appointments and exceptional database conditions.

4. Improved Strategy

- Draw the CFG and identify all decision points.
- Design tests for valid and invalid patient details.
- Test both branches of every condition.
- Test database errors and appointment conflicts.
- Measure statement and branch coverage and test important independent paths.
- Conduct regression testing after corrections.

Conclusion: White-box testing with suitable coverage criteria helps reveal hidden logic errors, improves reliability and reduces risks to patient care.

3. E-Commerce System – Testing Multiple Inputs

Introduction: The e-commerce system uses price, quantity and discount fields. Testing each field separately may fail to detect errors caused by combinations of inputs.

1. Equivalence Class Partitioning

Divide inputs into valid and invalid classes:

- Price: positive, zero and negative.
- Quantity: positive integer, zero and negative.
- Discount: valid percentage and invalid percentage outside 0–100%.

2. Boundary Value Analysis

Assume price must be positive, quantity ranges from 1 to 100, and discount ranges from 0% to 100%.

Input	Boundary test values
Price	0, minimum valid price, negative price
Quantity	0, 1, 2, 99, 100, 101
Discount	-1%, 0%, 1%, 99%, 100%, 101%

3. Decision Table Testing

Condition	Expected result
Valid price, quantity and discount	Calculate total
Invalid price	Reject input
Zero or negative quantity	Reject input
Discount below 0% or above 100%	Reject discount
Multiple valid inputs	Calculate correct discounted total

4. Revised Testing Strategy

- Apply ECP and BVA to individual fields.
- Use decision tables to test important input combinations.
- Check the calculation: Total = Price × Quantity × (1 − Discount/100).

- Test rounding, invalid values and maximum order quantities.
- Automate test cases and perform regression testing.

Conclusion: Combining ECP, BVA and decision table testing provides better input and combination coverage, improving price calculations and preventing incorrect orders.

4. Complex Module – Path Testing

Introduction: A software module with nested conditions and loops may contain many execution paths. Partial path coverage can leave defects undetected.

1. Control Flow Graph

Construct a CFG containing nodes for statements, decision points, loop entry, loop exit and program termination. Connect the nodes according to the possible execution flow.

2. Identify Independent Paths

Independent paths introduce at least one new edge not included in previously selected paths. Cyclomatic complexity estimates the number of linearly independent paths.

\[V(G)=E-N+2P \]

Where:

- \(E\) = number of edges.
- \(N\) = number of nodes.
- \(P\) = number of connected components, normally 1.

3. Testing Strategy

- Test every decision's true and false branches.
- Test each independent path identified from the CFG.
- Test loops with zero, one, two and multiple iterations, where applicable.
- Test nested conditions and loop termination.
- Check boundary conditions and invalid inputs.
- Review coverage reports and add tests for uncovered critical paths.

4. Expected Outcomes

This approach detects incorrect conditions, infinite loops, missing branches and incorrect termination logic. Since loops can produce many possible paths, prioritize independent paths and safety-critical paths rather than claiming exhaustive testing of every possible execution.

Conclusion: CFG-based path testing and cyclomatic complexity provide a systematic way to select test cases and improve defect detection.

5. Financial Application – Hybrid Testing Strategy

Introduction: Continued production defects indicate that combining black-box and white-box testing has not provided sufficient coverage or effective defect detection.

1. Black-Box Testing

- Apply ECP to valid and invalid transaction amounts.
- Apply BVA to minimum and maximum transaction limits.
- Test insufficient balance, invalid accounts and duplicate transactions.
- Verify expected outputs without depending on internal code.

2. White-Box Testing

- Construct a CFG of transaction processing logic.
- Test independent paths and decision outcomes.
- Verify balance checks, authorization, debit, credit and rollback logic.
- Measure statement and branch coverage.

3. Hybrid Testing Approach

Testing stage	Technique	Purpose
Input validation	ECP and BVA	Detect invalid and boundary inputs
Logic verification	CFG and path testing	Detect internal control-flow defects
Transaction testing	Integration testing	Verify debit and credit consistency
Final validation	Regression testing	Ensure fixes do not introduce new defects

4. Test Adequacy Criteria

Coverage percentages alone cannot prove that the application is defect-free. Review requirements coverage, branch coverage, critical paths, error handling and transaction integrity.

5. Justification

Black-box testing detects incorrect external behavior, while white-box testing identifies untested internal logic. Together, they cover more failure conditions than either approach alone.

Conclusion: A risk-based hybrid strategy with automation, coverage analysis and regression testing reduces financial errors and improves application reliability.

6. HR Payroll System – ECP and BVA

Introduction: The HR payroll system calculates bonuses based on years of service and performance ratings. Errors at exactly 5, 10 and 15 years suggest that boundary conditions may have been implemented incorrectly.

1. Equivalence Class Partitioning (ECP)

Divide years of service into classes based on the bonus rules. For example, if bonus levels change at 5, 10 and 15 years, create separate classes for each service interval. Divide performance ratings into valid and invalid categories.

2. Boundary Value Analysis (BVA)

Test the following service values:

Boundary	Test values
5 years	4, 5, 6
10 years	9, 10, 11
15 years	14, 15, 16

Also test the minimum and maximum allowed performance ratings and invalid ratings.

3. Improved Test Design

- Verify bonus calculations at every service boundary.
- Test different performance ratings within each service class.
- Check employees with incomplete years of service if the system supports them.
- Compare calculated bonuses with expected payroll values.
- Perform regression testing after fixing calculation errors.

4. Why ECP and BVA Are Better Than Exploratory Testing Alone

Exploratory testing depends on the tester's choices and may overlook exact boundaries. ECP systematically selects representative inputs, while BVA targets values where errors commonly occur. Exploratory testing can still supplement these techniques.

Conclusion: ECP and BVA provide systematic coverage of service intervals and performance ratings, helping ensure accurate bonus payments.

7. Temperature Control System – CFG and MC/DC

Introduction: The laboratory temperature control system uses if-else conditions to operate heaters, coolers and alarms. A missed alarm during sensor malfunction indicates insufficient decision coverage.

1. Sample Control Flow Graph

Start / Read sensors
Sensor functioning?

Yes

Check temperature
Control heater/cooler

No

Trigger alarm
Safe shutdown

2. Modified Condition/Decision Coverage (MC/DC)

MC/DC ensures that each individual Boolean condition can independently change the outcome of a decision while the other relevant conditions remain fixed.

For a decision:

Alarm = SensorFault OR (TemperatureHigh AND CoolingFailure)

Use the following tests:

Test	Sensor fault	Temperature high	Cooling failure	Alarm
1	False	False	False	False
2	True	False	False	True
3	False	True	False	False
4	False	True	True	True

These tests demonstrate the independent effect of each condition on the alarm decision.

3. Improved Testing Strategy

- Draw the CFG and identify every decision.
- Test true and false outcomes for each branch.
- Design MC/DC test pairs for every Boolean condition.
- Test sensor disconnection, overheating and cooling failure.

- Verify alarm activation and safe shutdown.
- Automate tests and review coverage reports.

Conclusion: MC/DC is more rigorous than statement coverage because it checks the independent influence of logical conditions. It improves fault detection in safety-critical temperature control systems.

8. Travel Booking Portal – Decision Table Testing

Introduction: Applying multiple promotional codes may produce a negative total amount. Testing each input independently is insufficient because the error can occur when price, promo type and user tier interact.

1. Identify Conditions

- Price: positive, zero or invalid.
- Promo type: percentage, fixed discount or incompatible promotion.
- User tier: regular, silver or premium.
- Promotion combination: allowed or not allowed.

2. Decision Table

Assume the system allows only eligible promotions and never permits a negative payable amount.

Rule	Valid price	Eligible promo	Eligible user tier	Expected result
R1	Yes	Yes	Yes	Apply permitted discount
R2	Yes	No	Yes	Reject promo
R3	Yes	Yes	No	Reject tier-restricted promo
R4	Yes	Multiple compatible	Yes	Apply according to policy
R5	Yes	Incompatible combination	Any	Reject combination
R6	No	Any	Any	Reject booking
R7	Yes	Discount exceeds price	Any	Cap or reject according to policy; total must not be negative

3. Cause-Effect Graphing

Map relationships between input conditions and outcomes. For example, a valid price and eligible promotion produce a discount, while an invalid price causes rejection. Convert the graph into test cases.

4. Optimized Test Set

- Test each valid and invalid condition.
- Test compatible and incompatible promo combinations.

- Test discounts equal to, below and above the booking price.
- Test different user tiers and their eligibility rules.
- Remove redundant cases that exercise the same rule without adding coverage.

5. Expected Results

The system should reject invalid combinations, calculate discounts correctly and prevent negative totals. The exact promotion rules must follow the portal's business requirements.

Conclusion: Decision table testing and cause-effect graphing systematically cover interacting conditions while reducing redundant test cases.

9. Database Migration Tool – Path Testing

Introduction: High statement coverage has not prevented infinite-loop crashes in the database migration tool. Statement coverage only confirms that statements execute; it does not verify all loop conditions or termination behavior.

1. Why Statement Coverage Is Insufficient

A test may execute every statement once but fail to test:

- Loop termination conditions.
- Zero or one iteration.
- Nested-loop interactions.
- Invalid records and exceptional conditions.
- Incorrect counter updates that cause infinite loops.

2. Path Testing Strategy

Construct a CFG for record validation and identify independent paths. Test the following cases:

1. Empty database or zero records.
2. One valid record.
3. Multiple valid records.
4. Invalid records requiring rejection.
5. Maximum supported dataset.
6. Nested loops with zero, one and multiple iterations.
7. Error conditions and loop exit behavior.

3. Cyclomatic Complexity

Calculate complexity using:

\[V(G)=E-N+2P \]

For one connected CFG, \(P=1\), so:

\[V(G)=E-N+2 \]

The result estimates the number of linearly independent paths needed for basis path testing.

4. Improved Strategy

- Draw the CFG and calculate its cyclomatic complexity.
- Design tests for each independent path.

- Verify that loop counters change correctly.
- Add timeout protection to detect non-terminating executions.
- Test loop boundaries and nested-loop exit conditions.
- Check record counts and data consistency before and after migration.

Conclusion: Basis path testing combined with loop-boundary testing detects termination defects that statement coverage can miss, improving migration reliability.

10. Food Delivery App – State Transition Testing

Introduction: The food delivery app moves through Ordered, Preparing, Out for Delivery and Delivered states. Driver cancellation can leave the app stuck because the cancellation transition may not be handled correctly.

1. State Transition Model

Ordered
Preparing
Out for Delivery
Delivered
Driver cancellation → Reassign driver or cancel order according to policy

2. State Transition Test Cases

Current state	Event	Expected result
Ordered	Restaurant accepts	Preparing
Preparing	Order ready and dispatched	Out for Delivery
Out for Delivery	Successful delivery	Delivered
Out for Delivery	Driver cancels	Reassignment or cancellation workflow
Delivered	Duplicate delivery event	Remain Delivered
Cancelled	Delivery event received	Reject invalid transition

3. Improved Testing Strategy

- Test every valid state transition.
- Test invalid transitions, such as Delivered to Preparing.
- Test driver cancellation at different stages.
- Verify notifications, refunds and driver reassignment.
- Test duplicate and delayed network events.
- Confirm that the application does not remain in an undefined state.

4. Why State Transition Testing Is Necessary

ECP and BVA test input categories and numerical limits, but they may miss errors caused by event sequences and changes in application state. State Transition Testing directly addresses these workflow problems.

Conclusion: State Transition Testing verifies both valid and invalid transitions, ensuring consistent tracking and reliable cancellation handling.

11. Cybersecurity Authentication – Hybrid Testing

Introduction: The authentication module uses password strength, OTP timing and IP location. Brute-force vulnerabilities indicate weaknesses in authentication logic, attempt limits or security controls.

1. Black-Box Testing

Test the module from the user's perspective:

- Correct and incorrect passwords.
- Valid, invalid and expired OTPs.
- Repeated login attempts.
- Login attempts from unfamiliar IP addresses.
- Account lockout and rate limiting.
- OTP reuse and repeated requests.

2. White-Box Testing

Construct a Control Flow Graph (CFG) of the authentication process. Identify independent paths through password validation, OTP verification, IP checks and access granting.

Test paths such as:

1. Correct password and valid OTP.
2. Incorrect password.
3. Correct password but expired OTP.
4. Suspicious IP address.
5. Repeated failed attempts.
6. Multiple failures followed by a correct OTP.

3. Error Guessing

Use experience to predict likely defects, such as missing lockout controls, OTP reuse, weak rate limits and incorrect session handling.

4. Hybrid Testing Strategy

- Apply black-box tests to authentication inputs and responses.
- Use error guessing to identify security weaknesses.
- Apply CFG and path analysis to examine internal decision logic.

- Verify rate limiting, lockout and audit logging.
- Perform regression and security testing after fixes.

Conclusion: Combining black-box testing, error guessing and white-box path analysis improves authentication security and reduces the risk of brute-force attacks. It should be supplemented with dedicated security testing.

12. E-Learning Platform – Equivalence Class Partitioning

Introduction: The e-learning platform provides Free, Basic and Premium subscriptions. Errors occur when users upgrade exactly on the expiration date, suggesting inadequate testing of date boundaries and subscription transitions.

1. Equivalence Class Partitioning

Divide subscription dates into classes:

- Before expiration.
- Exactly on the expiration date.
- After expiration.
- Invalid dates or missing subscription dates.

Divide subscription types into Free, Basic and Premium. Also test valid and invalid upgrade combinations.

2. Boundary Test Cases

Assume a subscription expires on 31 October.

Test case	Date or condition	Expected result
Before expiration	30 October	Existing access remains valid
On expiration date	31 October	Apply the defined expiration policy
After expiration	1 November	Expired access follows system rules
Upgrade before expiration	Valid upgrade request	Correct upgraded access
Upgrade on expiration date	31 October	Correct upgrade and billing
Upgrade after expiration	1 November	Apply renewal or reactivation rules

The expected behavior on the expiration date must follow the platform's documented policy and timezone rules.

3. Improved Test Strategy

- Apply ECP to dates and subscription categories.
- Test upgrades and downgrades between all supported plans.

- Verify access permissions for each course type.
- Check billing, renewal dates and expiration notifications.
- Test leap years, month-end dates and timezone differences where applicable.
- Perform regression testing after corrections.

Conclusion: ECP provides systematic coverage of subscription categories, while boundary-focused tests expose date-related defects. Together, they support seamless subscription transitions.

13. Smart-Home Lighting – Branch and Condition Coverage

Introduction: The smart-home lighting controller uses switch-case statements and Boolean expressions to adjust brightness according to ambient light. Statement coverage alone may miss errors in individual conditions and decision outcomes.

1. Limitations of Statement Coverage

Executing every statement does not guarantee that every branch or Boolean condition has been tested. Some logical errors may remain hidden even when all statements execute.

2. Branch Coverage

Branch coverage requires each decision outcome to execute at least once. For example, test ambient light levels that cause the controller to increase brightness and decrease brightness.

3. Condition Coverage

Condition coverage requires each individual Boolean condition to evaluate to both true and false.

Consider:

IncreaseBrightness = (AmbientLight < Threshold) AND (ManualMode = False)

Test	Ambient light below threshold	Manual mode	Expected behavior
1	True	False	Increase brightness
2	False	False	Do not increase
3	True	True	Do not increase

These tests exercise both outcomes of each condition.

4. Improved White-Box Strategy

- Construct a CFG for switch-case and conditional logic.
- Test every switch-case option and default branch.
- Achieve branch and condition coverage.
- Test sensor failures and extreme light readings.
- Verify the actual light response through hardware integration tests.
- Perform regression testing after changes.

Conclusion: Branch and condition coverage reveal logical errors missed by statement coverage. Hardware integration

testing further verifies that software decisions correctly control physical lights.

14. Logistics System – Robust Boundary Value Analysis

Introduction: The logistics system calculates shipping costs based on weight, destination zone and delivery speed. Failure to reject zero or negative weights indicates insufficient input validation.

1. Robust Boundary Value Analysis

Assume the minimum permitted weight is greater than zero and the maximum permitted weight is $W_{\max}$.

Test case	Input weight	Expected result
Invalid lower value	-1	Reject
Invalid zero value	0	Reject
Minimum valid value	$W_{\min}$	Accept
Just above minimum	$W_{\min}+\delta$	Accept
Just below maximum	$W_{\max}-\delta$	Accept
Maximum valid value	$W_{\max}$	Accept
Above maximum	$W_{\max}+\delta$	Reject

Here, δ represents the smallest supported weight increment. Exact limits must follow the system requirements.

2. Additional Test Cases

- Invalid destination zone.
- Unsupported delivery speed.
- Missing or non-numeric weight.
- Very large weights.
- Valid weights combined with different zones and delivery speeds.

3. Improved Testing Strategy

- Apply robust BVA to valid and invalid boundaries.
- Use ECP to classify acceptable and unacceptable inputs.

- Verify that invalid weights are rejected before cost calculation.
- Check shipping charges for valid combinations.
- Automate negative tests and regression tests.

4. Importance of Negative Testing

Negative testing verifies that the system handles invalid data safely. Without it, the system might produce negative charges, incorrect invoices or inconsistent shipping calculations.

Conclusion: Robust BVA and negative testing improve input validation, prevent incorrect shipping costs and increase system reliability.

15. Medical Imaging Software – Basis Path Testing

Introduction: The medical imaging software processes large datasets using parallel loops. Inconsistent results for prime-numbered data sizes suggest defects in loop boundaries, data partitioning or parallel execution.

1. Limitations of Random Testing

Random testing may not consistently select prime-sized datasets or reproduce concurrency-related failures. Therefore, a structured white-box strategy is required.

2. Basis Path Testing

- Construct a CFG of the image-processing algorithm.
- Identify decision points and independent execution paths.
- Calculate cyclomatic complexity:

\[V(G)=E-N+2P \]

For a single connected graph, \(V(G)=E-N+2\). This estimates the number of independent paths for basis path testing.

3. Test Data Design

Test different dataset sizes:

- Zero or empty input, if supported.
- One element.
- Small composite-number sizes.
- Small prime-number sizes, such as 7 and 11.
- Large prime-number sizes.
- Large composite-number sizes.

Also test different image dimensions, uneven partitions and parallel worker counts.

4. Improved Testing Strategy

- Test each independent path identified from the CFG.
- Examine loop initialization, termination and index calculations.
- Verify that every data element is processed exactly as required.
- Compare parallel results with a trusted sequential implementation.
- Repeat tests with different worker counts to detect race conditions.
- Check numerical accuracy, memory usage and timeout behavior.

- Use regression testing after corrections.

5. Expected Outcomes

These tests can reveal incorrect loop limits, omitted data elements, duplicate processing and concurrency defects. Prime-sized datasets are especially useful because they may expose uneven partitioning problems.

Conclusion: Basis Path Testing, loop-boundary analysis and parallel-execution testing provide a structured approach to identifying hidden defects and improving the accuracy and stability of medical imaging software.

Exam tip: For each 10-mark answer, write the introduction, explain the relevant testing technique, include the test-case table or CFG where applicable, describe the improved strategy, and finish with a conclusion. Adapt the test values to the exact limits and rules specified in your question.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification